

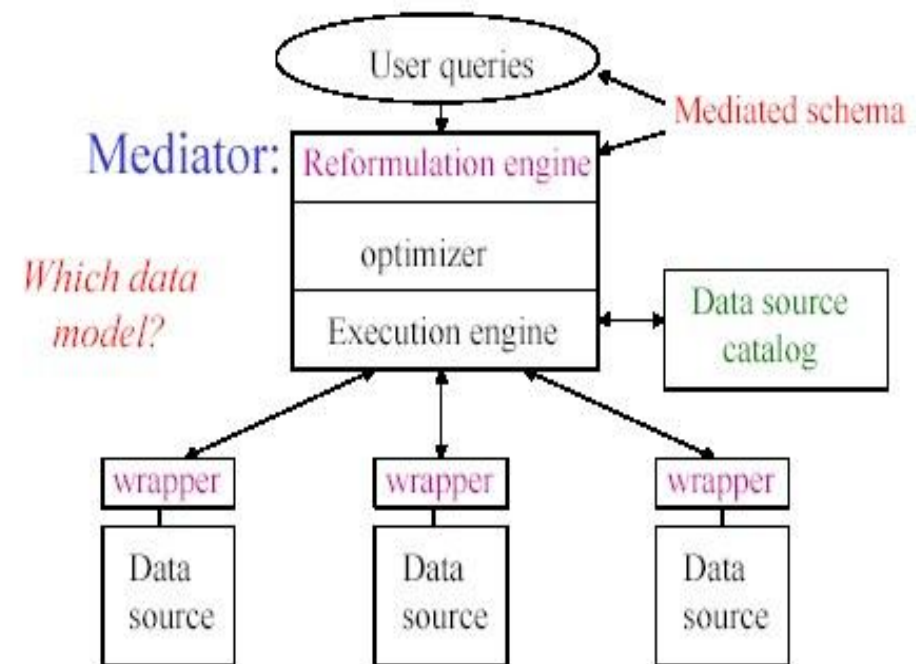
II Architecture: Virtualization Layer Approach

Virtual integration favours freshness and source autonomy over predictable performance.

Workflow:

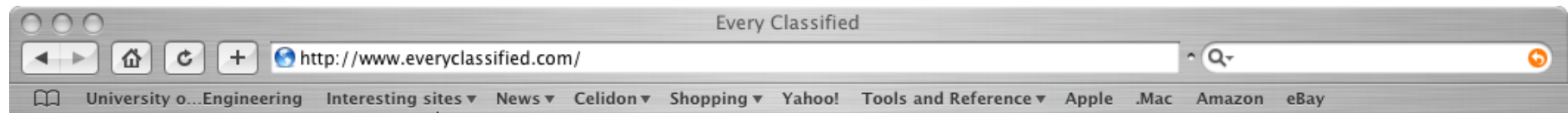
Query → decomposition → wrapper invocation → integration

- Leave the data in the sources.
- When a query comes in:
 - Determine the relevant sources to the query
 - Break down the query into sub-queries for the sources.
 - Get the answers from the sources, and combine them appropriately.
- Data is fresh. Approach scalable
- Issues:
 - Relating Sources & Mediator
 - Reformulating the query
 - Efficient planning & execution



Garlic [IBM], Hermes[UMD];Tsimmis,
InfoMaster[Stanford]; DISCO[INRIA];
Information Manifold [AT&T];
SIMS/Ariadne[USC];Emerac/Havasu[ASU]

Integration for creating a single site to search for jobs/rentals/...



 [About EveryClassified Search](#)

AUTOS



Research new and used makes and models to find the right car for you.

Make

Model

Year

Location
(Enter City, State or Zip Code)

RENTALS



Search for apartments or houses for rent.

Type

From to

Bed

Location
(Enter City, State or Zip Code)

REAL ESTATE



Make finding your next home easier. Search homes for sale.

From to

Bed Bath

Location
(Enter City, State or Zip Code)

JOBS



Find a job.

Keywords
(ie. Retail, Sales, Management)

Location
(Enter City, State or Zip Code)

PERSONALS



Find that special someone.

I am a

seeking a

Age to

Location
(Enter City, State or Zip Code)

GENERAL



Search for miscellaneous items for sale.

Keywords
(ie. furniture, Wood, Bookshelf)

Location
(Enter City, State or Zip Code)

[Autos](#) | [General](#) | [Jobs](#) | [Personals](#) | [Real Estate](#) | [Rentals](#) | 2005 EveryClassified [Submit a Site](#) Created by 

Data Source Selection, Schema Matching and Mapping

Data Source Selection

- **Once you have the set of questions for querying integrated data using the mediator approach, the next step is to identify the data sources [This is also true for other approaches of II]**
- **Must evaluate the data sources to know which to select**
 - Is it complete? In terms of coverage? In terms of missing data?
 - Is it correct?
 - Is it clean?
 - Is it structured? If not, can you extract structured data out of it easily and accurately or expose text document as an attribute?
 - Is it up to date? How quickly is it changing?
 - Hard to answer some of these questions until you have acquired some data from the sources

Data Acquisition

- Then need to acquire the data from the sources

- This is highly non-trivial
- Types of data sources:
 - **Structured data sources:** relational databases, XML files, ...
 - **Textual data:** emails, memos, documents, etc.
 - **Web data:** need to crawl, maybe can get a dump, API may exist
 - **Other types of data:** excel, pdf, images, etc.
- Being able to extract data from such sources is non-trivial, time consuming
- Build connectors, wrappers,...
- **Some of the data come from within the company**
 - Need to talk with data owner, convincing him/her, get help to get the data
 - Can take months – acquisition due to legal and compliance reasons.
- **Some of the data come from outside the company**
 - Public data, Open source data, Paid data
 - Pros: clean, quick
 - Cons: untrustworthy, noisy, expensive

Understanding, Cleaning, & Transformation

Do These for Each Source, then Integrate

- **For data from each source**

- **Data problems**

- missing values, incorrect values, illegal values, outliers, misspellings
 - Synonyms, conflicting data (eg, age and birth year)
 - variations of values, duplicate tuples, wrong value formats
 - understand current vs ideal schema/data
 - Attribute values profiling, relationship between attributes, integrity constraints...
 - Tools exist for data profiling, relationship discovery,...
 - Identify violations of constraints, if any, for the schema/data
 - clean and transform
 - possibly enrich/enhance

- **Integrate data from the multiple sources**

- schema matching/merging, data matching/merging
 - misspelt names
 - violating constraints (key, uniqueness, foreign key, etc)

Matching discovers correspondences; mapping operationalizes them into executable integration logic.

Schema matching vs schema mapping

Schema matching asks

- Which elements correspond?
- Does customer_name match client_full_name?
- Can one source attribute map to several target concepts?
- How strong is the correspondence?

Schema mapping asks

- How do we express the correspondence formally?
- What transformation is required?
- Can the mapping be executed?
- Does it require joins, filters, aggregation or functions?
- How can queries be rewritten?

- Input: Two or more schemas $\Sigma_1, \Sigma_2, \dots, \Sigma_n$.
- Output: A set of correspondences:
 - $M = \{(a_i, b_j, \text{sim}(a_i, b_j)) \mid a_i \in \Sigma_1, b_j \in \Sigma_2\}$
- Objective: Identify semantically related attributes, i.e.
 - $\text{sim}(a, b) \in [0, 1]$
 - Match if $\text{sim}(a, b) \geq \theta$

Matching is discovery; mapping is an executable semantic relationship.

Why schema matching is hard

Common semantic problems

Synonyms (Naming Conflict): `customer_name` vs `client_name`

Acronyms: DOB = Date of Birth

Granularity: `full_name` vs `first_name` + `last_name`

Structural mismatch: nested relations vs tables

Derived attributes: `age` vs `date_of_birth`

Temporal mismatch: current vs historical department

Semantic Conflicts: `NetPrice` = `GrossPrice` - `Discount`

Representation Conflicts: `Date` = DD/MM/YYYY **VS** YYYY-MM-DD

Evidence beyond names

Data types and constraints

Value distributions / examples

Neighbouring attributes

Keys and foreign keys

Table/class context

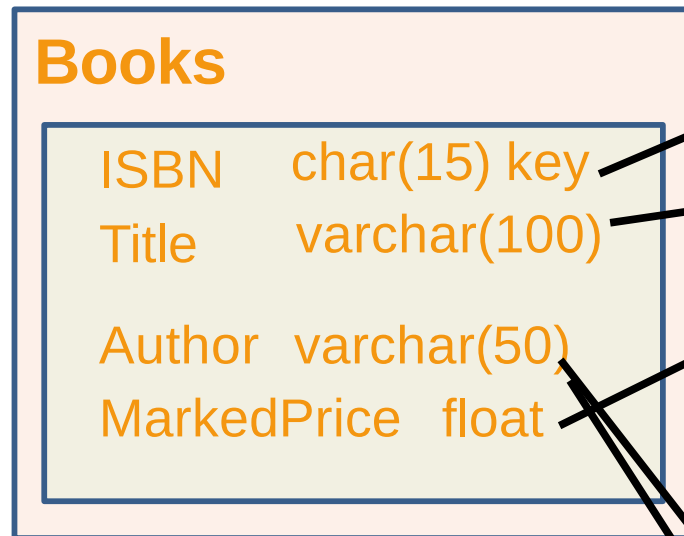
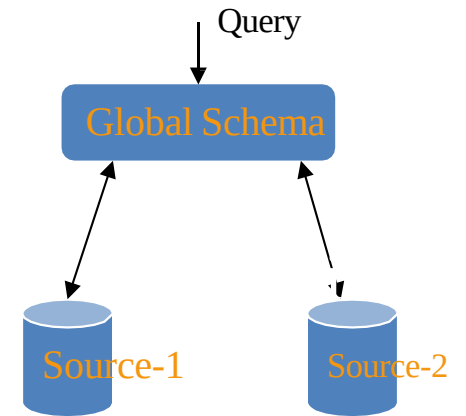
Business glossary / ontology

Source ownership and provenance

A strong matcher uses names + structure + instances + constraints + context.

The Schema Matching Problem

Given two input schemas in any data model and, optionally, auxiliary information, compute a mapping between schema elements of the two input schemas that passes user validation.



BookInfo

ID	char(15) Primary Key
AuthorID	integer references AuthorInfo
BookTitle	varchar(150)
ListPrice	float
DiscountPrice	float

AuthorInfo

AuthorID	integer key
LastName	varchar(25)
FirstName	varchar(25)

$\pi_{\text{ISBN, Title, MarkedPrice}}(\text{Books})$

$= \pi_{\text{ID, BookTitle, ListPrice}}(\text{BookInfo})$

$\pi_{\text{Author}}(\text{Books}) = \pi_{\text{FirstName+LastName}}(\text{AuthorInfo})$

$\text{Books} = \pi_{\text{ID, BookTitle, FirstName+LastName, ListPrice}}(\text{BookInfo} \bowtie \text{AuthorInfo})$

Inputs to Matching Technique

- Schema structure
- Attribute names
- Synonyms
 - Code = Id = Num = No
 - Zip=PIN = Postal [code]
 - Node = Server
- Acronyms
 - PO = Purchase Order
 - UOM = Unit of Measure
 - SS# = Social Security Number
- Data instances
 - Attributes match if they have similar instances or value distributions

Similarity Functions

- Lexical (Linguistic)-based: Edit distance, Jaccard, cosine (token-level), synonym expansion
- Semantic-based: WordNet, ontologies, embeddings.
- Structural-based: Uses schema tree/graph
- Instance-based: Distribution similarity (e.g., KL divergence).
- Hybrid:

$$sim(a, b) = \alpha \cdot sim_{lex}(a, b) + \beta \cdot sim_{sem}(a, b) + \gamma \cdot sim_{inst}(a, b)$$

Linguistic/Lexical Matching

- Tokenization of names
 - PurchaseOrder → purchase + order
- Expansion of acronyms
 - UOM → unit + of + measure
- Clustering based on keywords and data-types
 - Street, City, POAddress → Address
- Linguistic similarity
 - Pair-wise comparison of elements that belong to the same cluster
 - Token similarity = $f(\text{string matching, synonyms score})$
 - Token set similarity = average (best matching token similarity)
- Thesaurus: acronymns, synonyms, stop words and categories

Example – Linguistic/Lexical Matching

Schema 1: Customer(CustID, Name, Address)

Schema 2: Client(CustomerNumber, FullName, Location)

Similarities:

CustID $\leftrightarrow$ CustomerNumber = 0.92 (edit distance).

Name $\leftrightarrow$ FullName = 0.88 (cosine).

Address $\leftrightarrow$ Location = 0.73 (Jaccard).

Threshold $\theta = 0.8 \rightarrow$ first two accepted as matches.

Example – Semantic Matching

Schema 1: Invoice(TotalPrice)

Schema 2: Order(GrossPrice, Discount)

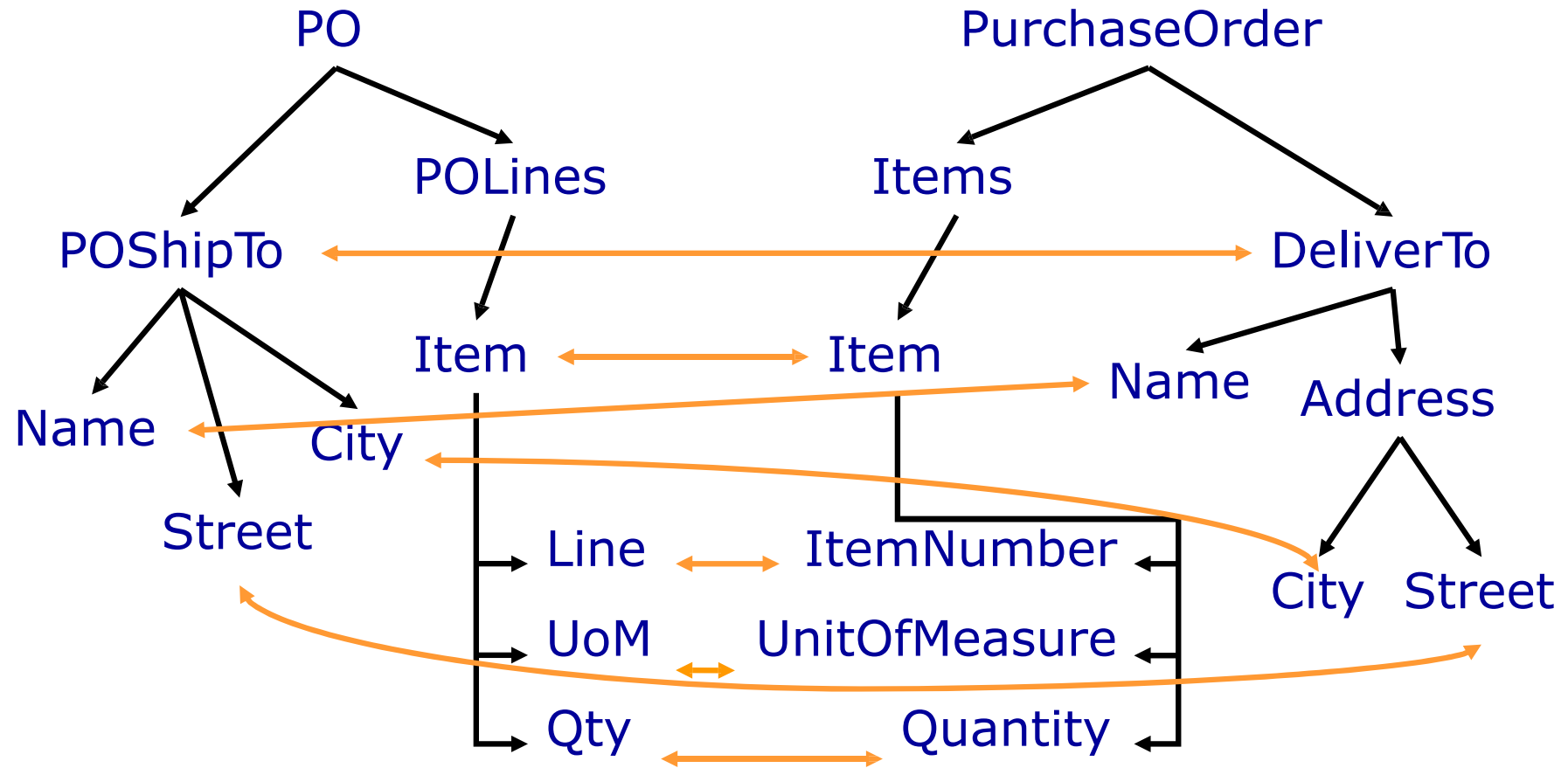
Ontology rule:

$$\textit{TotalPrice} = \textit{GrossPrice} - \textit{Discount}$$

Transformation rule derived (Mapping between Schema 1 and 2):

```
SELECT (GrossPrice – Discount) AS TotalPrice FROM Order;
```

Structure Matching



Tree Match Algorithm (Bottom-up Traversal)

- Atomic elements (leaves) are similar if
 - Linguistically and data-type similar
 - Their contexts, i.e., ancestors, are similar
- Compound elements (non-leaves) are similar if
 - Linguistically similar
 - Elements in their context, i.e., subtrees rooted at the elements, are similar
- Mutually dependent formulation
 - Leaves determine internal node similarity
 - Similarity of internal nodes leads to increase in leaf similarity

Schema-based hybrid matching algorithm

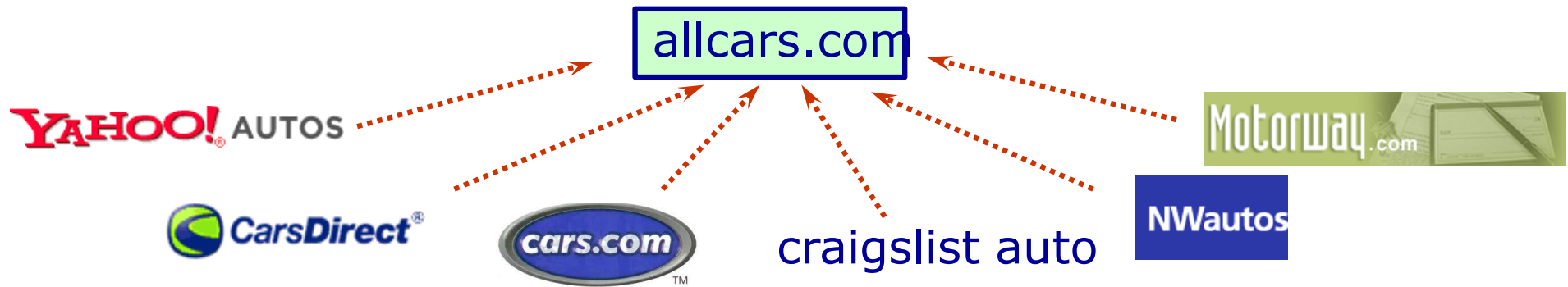
Based on combining multiple approaches that use only schema (no instances)

Input: Two schema graphs

Output: Similarity matrix and candidate mapping

- **Linguistic matching:** compare attributes based on names, data types, etc
 - Use a thesaurus to help match names by identifying short-forms (Qty for Quantity), acronyms (UoM for UnitOfMeasure) and synonyms (Bill and Invoice). The result is a linguistic similarity coefficient, L_{sim} , between each pair of elements.
- **Structure matching:** compare elements based on the similarity of their contexts or vicinities. The result is a structural similarity coefficient, S_{sim} , for each pair of elements.
- **Compute the Weighted similarity:** $W_{sim} = w * L_{sim} + (1 - w) * S_{sim}$
- **Mapping generation:** a mapping is created by choosing pairs of schema elements with maximal weighted similarity.

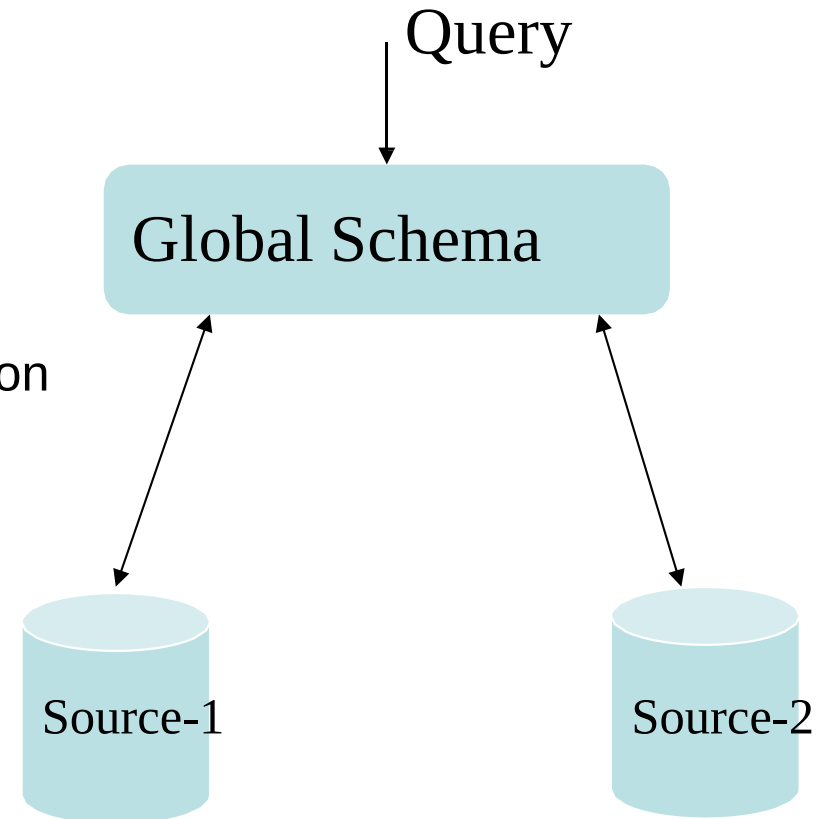
Collective Schema Matching



[He+, SIGMOD'03]: Build mediated schema for a domain by clustering elements in multiple schemas

Schema Mapping

- Global schema defined in terms of sources (global schema centric or Global-As-View (GAV))
 - Query reformulation easier
 - Any change in sources schema, needs change in global schema
 - Global relations cannot model any information not present in at least one source.
- Sources defined in terms of global schema (source-centric or Local-As-View (LAV))
 - High modularity and extensibility (if the global schema is well designed, when a source changes, only its definition is affected)
 - Query reformulation complex
 - It allows to add a source to the system independently of other sources.



Mapping Rule Formalism and Composition

- Given $a \in \Sigma_1$, $b \in \Sigma_2$.
- Transformation function:
 - $b = f(a)$
- Examples of f :
 - Identity: $b = a$.
 - Scaling: $b = a \times c$.
 - Derived: $b = a_1 + a_2$.
- If $M_{12} : \Sigma_1 \rightarrow \Sigma_2$ and $M_{23} : \Sigma_2 \rightarrow \Sigma_3$, then:
 - $M_{13} = M_{23} \circ M_{12}$
- Enables chaining transformations.
- Example: CSV $\rightarrow$ JSON $\rightarrow$ Relational DB.

Example

Mediated schema:

Movie(title, dir, year, genre),
Schedule(cinema, title, time).

Express mediator schema
relations as views over
source relations

[S1(title,dir,year,genre)]

[S2(title, dir,year,genre)]

[S3(title,dir), S4(title,year,genre)]

Example taken from Dr. Subbarao Kambhampati's lecture notes.

Global-as-View (GAV)

Mediated schema:

Movie(title, dir, year, genre),
Schedule(cinema, title, time).

Express mediator schema
relations as views over
source relations

Create View Movie AS

select * from S1 [S1(title,dir,year,genre)]

union

select * from S2 [S2(title, dir,year,genre)]

union

[S3(title,dir), S4(title,year,genre)]

select S3.title, S3.dir, S4.year, S4.genre

from S3, S4

where S3.title=S4.title

Mediator schema relations are
Virtual views on source relations

Local-as-View (LAV)

Mediated schema:

Movie(title, dir, year, genre),
Schedule(cinema, title, time).

Express source schema
relations as views over
mediator relations

Create Source S1 AS

select * from Movie

S1(title,dir,year,genre)

Create Source S3 AS

select title, dir from Movie

S3(title,dir)

Create Source S5 AS

select title, dir, year
from Movie

S5(title,dir,year), year > 1960

where year > 1960 AND genre="Comedy"

Sources are “materialized views” of
mediator schema

GAV vs. LAV

Mediated schema:

Movie(title, dir, year, genre),
Schedule(cinema, title, time).

Source S4: S4(cinema, genre)

Create View Movie AS

```
select NULL, NULL, NULL, genre
from S4
```

Create View Schedule AS

```
select cinema, NULL, NULL
from S4.
```

But what if we want to find which cinemas are playing comedies?

Create Source S4

```
select cinema, genre
from Movie m, Schedule s
where m.title=s.title
```

Now if we want to find which cinemas are playing comedies, there is hope!

Lossy mediation

GAV

vs.

LAV

- Not modular
 - Addition of new sources changes the mediated schema
- Can be awkward to write mediated schema without loss of information
- Query reformulation easy
 - *reduces to view unfolding (polynomial)*
 - Can build hierarchies of mediated schemas
- Best when
 - Few, stable, data sources
 - well-known to the mediator (e.g. corporate integration)
 - Garlic, TSIMMIS, HERMES

- Modular--adding new sources is easy
- Very flexible--power of the entire query language available to describe sources
- Reformulation is hard
 - Involves answering queries only using views
- Best when
 - Many, relatively unknown data sources
 - possibility of addition/deletion of sources
 - Information Manifold, InfoMaster, Emerac, Havasu

Q1: The 'BookStore.com' is an aggregator that provides a single stop solution to their clients for buying books. The business has agreement with three suppliers, and the schema of each supplier is given below. Note, each supplier may have a different price for the same book. The primary key in each schema is underlined.

Supplier1(ISBN, Book_Name, Book_Price)

Supplier2(ISBN, BTitle, BPrice)

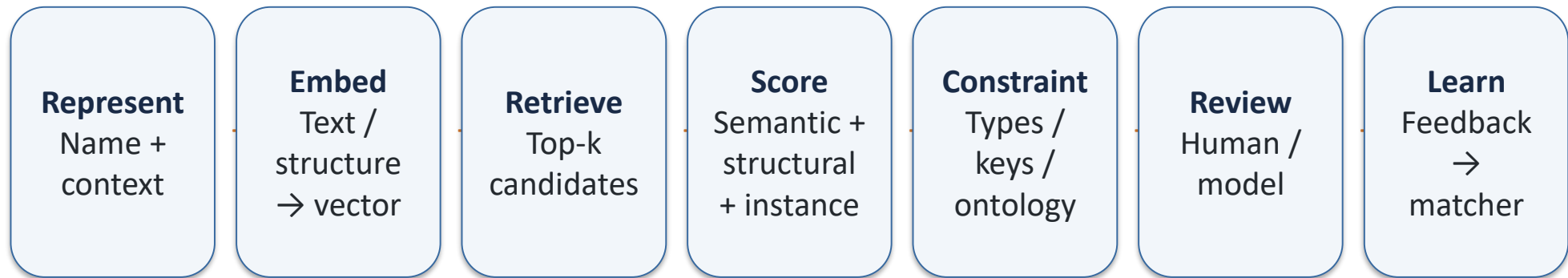
Supplier3(ISBN, Book_Title, Price)

- a. Design the global schema of the BookStore.com where (1) the client can create his/her profile (Name, Home#, St-Name, Area, City, PIN, Mobile), (2) the client can view/search all his/her current and past transactions (Order#, Date, ISBN, Book-Title, Price, Supplier-Name), and (3) the client can search the books availability by <title> and/or <ISBN> and can also view the supplier name.
- b. Write the schema mapping between the global schema (obtained in part (a)) and local schemas using GAV.
- c. The BookStore.com decides to show the supplier name selling the book at the lowest price among all, that is, 1 record per ISBN unless more than one supplier has the same lowest price. this case, it displays information of all suppliers. For this constraint, design the schema and its mapping to global schema (obtained in part (a)).

ML/DL based Algorithmic Approaches

1. Rule-based Matching: handcrafted rules.
2. Machine Learning: supervised or semi-supervised classifiers.
 - a. Input: feature vector $f(a, b)$.
 - b. Output: match / non-match probability.
3. Deep Learning: embedding-based similarity (BERT, graph embeddings).

AI/ML-driven schema matching pipeline



THINK

Why should embeddings primarily generate candidates rather than directly decide equivalence?

What should be embedded?

- Attribute name and description.
- Table/schema context and neighbouring attributes.
- Representative data values and distributions.
- Keys, foreign keys and relationship structure.
- Ontology class/property, domain and range.
- Business glossary, source owner and temporal context.

The best representation embeds context, not just the column name.

Embedding representation — example

Weak representation

```
embed('cust_nm')
```

Very little semantic context.

Possible candidates:

- customer_name
- customer_number
- customer_status
- party_name

Context-rich representation

```
embed('Attribute: cust_nm  
Table: CUSTOMER  
Meaning: unique customer name  
Type: VARCHAR  
Examples: Anil Kumar, Sarah Lee  
Neighbours: cust_id, email')
```

Context-rich embeddings reduce ambiguity because the model sees meaning, type, examples and relational context.

Candidate generation with embeddings

Step 1 — vector retrieval

- Encode source attribute as e_s .
- Encode target attributes as e_t .
- Compute cosine similarity:
- $\text{sim}(e_s, e_t) = (e_s \cdot e_t) / (||e_s|| ||e_t||)$
- Retrieve top-k target candidates.

Step 2 — validate/rerank

- Datatype compatibility.
- Value distribution similarity.
- Structural-neighbour similarity.
- Key / foreign-key compatibility.
- Ontology domain/range.
- Cross-encoder/LLM/human review.

Embedding similarity is a retrieval signal—not semantic proof.

Concrete embedding example

Source attribute

`SIS_STUDENT.full_name`

Type: VARCHAR

Examples: Anil Kumar, Sarah Lee

Neighbours: sid, mail

Table: student master data

Candidates

`STUDENT.name` → 0.94

`INSTRUCTOR.name` → 0.91

`DEPARTMENT.dept_name` → 0.62

Embedding alone cannot
distinguish the first two.

Structural context and constraints break the tie.

Multi-signal matching model

Feature signals

- N = name/description similarity
- I = instance/value similarity
- S = structural similarity
- C = constraint/type compatibility
- O = ontology compatibility

Simple ranker

- Score = $w_N \cdot N + w_I \cdot I + w_S \cdot S + w_C \cdot C + w_O \cdot O$
- Weights may be hand-designed or learned.
- Use labelled correspondences for supervised ranking.
- Evaluate top-1 and top-k performance.

Research question: can the weights be learned across domains without losing explainability?

Entity-level extension — embeddings + evidence

Candidate pair

Student:

name = 'A. Kumar'

email = 'akumar@uni.edu'

Person:

name = 'Anil Kumar'

email = 'anil.kumar@uni.edu'

Decision evidence

name embedding = high

email/domain = strong

affiliation = compatible

coauthor/context = medium

→ high match probability

→ retain evidence + confidence

Schema matching and entity resolution can share representations, but they solve different integration problems.

Where embedding-based matching fails

Semantic traps

- Address vs billing_address: related, not equivalent.
- Revenue vs profit: related, not equivalent.
- Status: source-specific meaning.
- Course code vs course ID: similar labels, different semantics.

Engineering traps

- Poor documentation poisons embeddings.
- Rare values may be uninformative.
- Domain abbreviations may be unseen.
- Model bias/drift changes similarity.
- Thresholds do not transfer reliably.

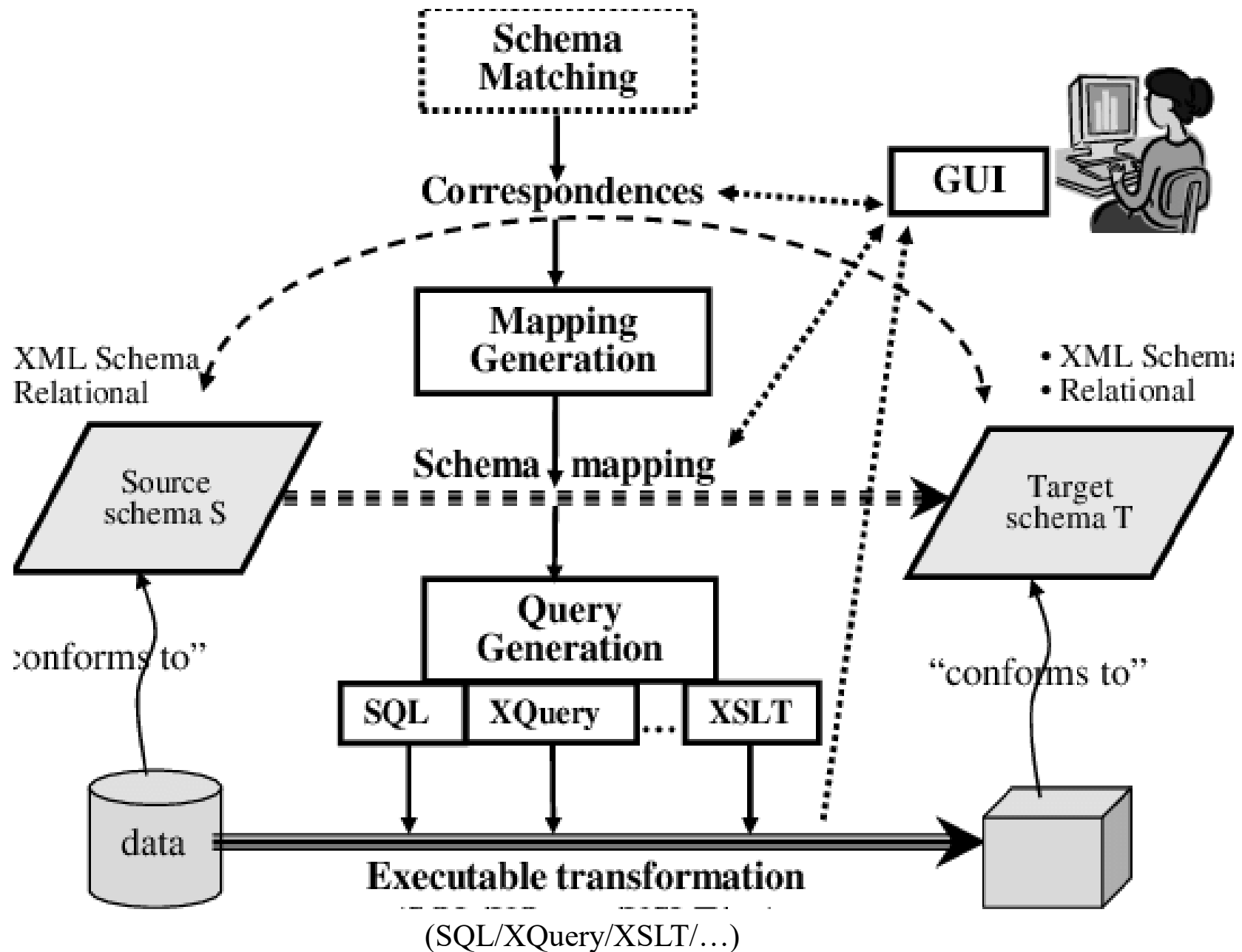
Use embeddings to accelerate discovery; use domain constraints and evidence to establish correctness.

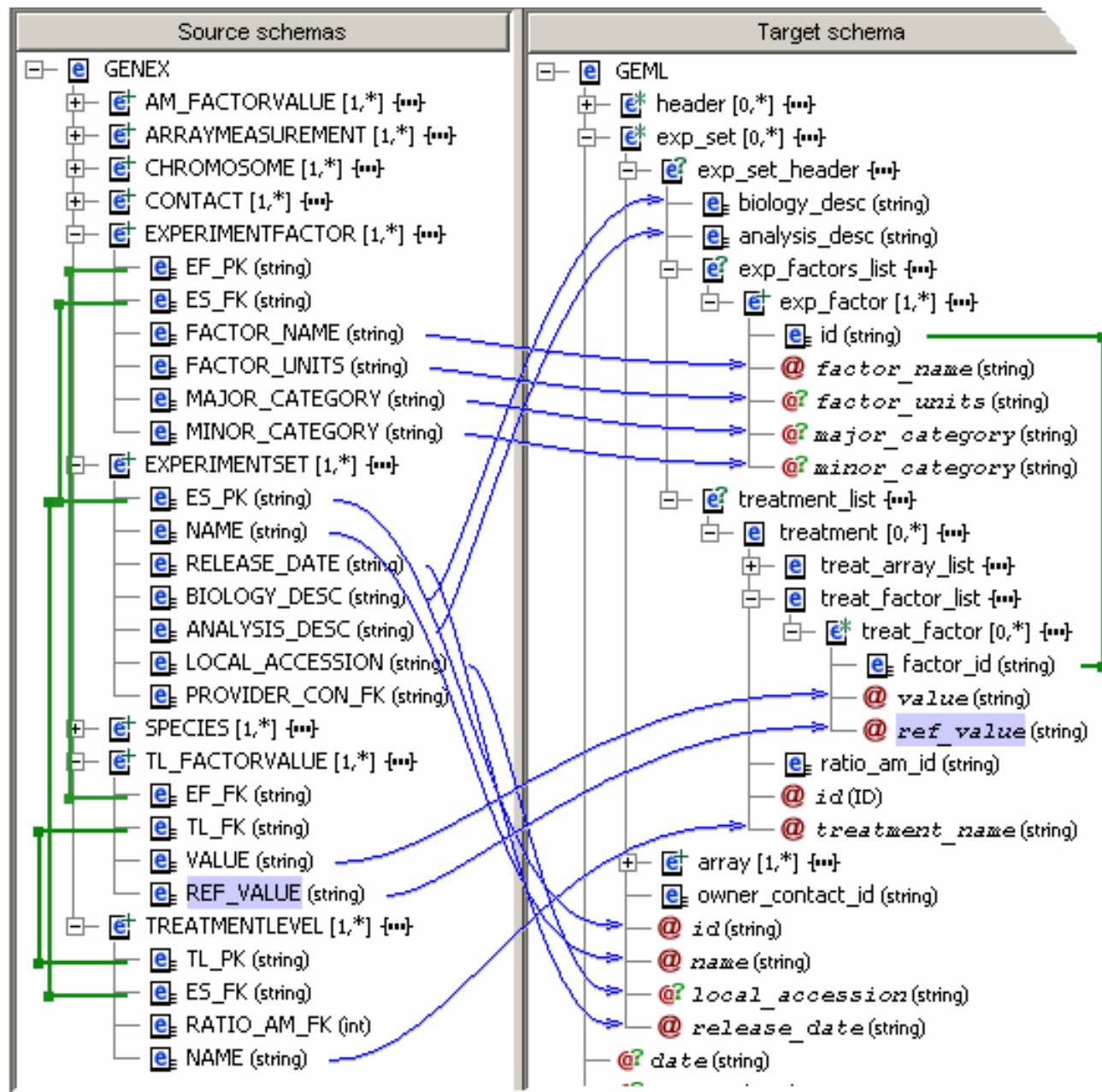
Tools & Frameworks

- Traditional: Clio, COMA++, Cupid.
- ML based: DeepMatcher, Magellan.
- ETL Systems: Informatica, Talend.
- Big Data: Apache NiFi, Spark SQL.

Clio: Schema Discovery and Mapping for Integration

CLIO Architecture





Takeaways

- Schema matching = fundamental for integration, NP-hard in general → heuristics & ML essential.
- Transformation rules enable data exchange. Mapping rules formalized via Datalog, SQL, or ontology mappings.
- Linguistic, structural, instance-based, hybrid approaches (semantic + ML/DL + rule-based) are state-of-art.
- Practical: often semi-automatic + domain expert validation.
- Key open problems: scalability, dynamic schema evolution, probabilistic mappings.